VAAL UNIVERSITY OF TECHNOLOGY

Inspiring thought. Shaping talent.

Faculty	:	Applied and Computer Sciences
Department	:	Computer Sciences
Subject Code	:	AIBUY3A
Lecturer	:	Dr Sibanda, Mr Matsela, Mr Matshego
Moderator	:	Mr XP Piyose
Year	:	2026

Assessment Name:	Documentation – PREDICT CARE
Assessment Date:	02 November 2026

STUDENT NUMBER	SURNAME AND INITIALS	SIGNATURE
240331001	Hlatshwayo K	
224494481	Skhosana L	
224954555	More A	
224051776	Rasilingwani M	
222113200	Kaniki M	
240234103	Hlophe L	
224158678	Mahloane M	
223958018	Mavuyangwa AS	

Declaration

We,

Khanyisile Hlatshwayo 240331001
Linda Skhosana 224494481
Atlegang More 224954555
Murendeni Rasilingwani 224051776
Michael Kaniki 222113200
Loyiso Hlophe 240234103
Moeketsi Mahloane 224158678
Angel Shiluva Mavuyangwa 223958018
declare that:

- The content of this project represents our own unaided work, and the project has not previously been submitted for academic examination towards any qualification.
- We have not engaged in plagiarism or used unauthorized materials in our work.
- All resources used in the work have been appropriately cited and acknowledged.
- This work represents our own opinions and not necessarily those of the Vaal University of Technology.

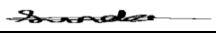
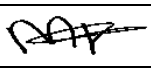
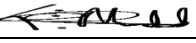
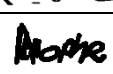
Full Name and Surname	**Signature**	**Date**
Khanyisile Hlatshwayo		2026/10/26
Linda Skhosana		2026/10/26
Atlegang More		2026/10/26
Murendeni Rasilingwani		2026/10/26
Michael Kaniki		2026/10/26
Loyiso Hlophe		2026/10/26
Moeketsi Mahloane		2026/10/26
Angel Shiluva Mavuyangwa		2026/10/26

Table of contents

DOCUMENTATION ASPECTS **1**
1. PREDICT CARE's relevance to the theme 1
2. Business objectives 1
3. Business success criteria 2
4. Business background 3
5. Requirements 4
6. Constraints 5
7. Risks 6
8. Tools and techniques 7
9. Problem definition 8

THEORETICAL ASPECTS **9**
1. Machine learning approach 9
2. Data 9
3. Model development and evaluation 10
4. Time-series analysis on data 11
5. Solution techniques and main scripts 12
6. Natural language processing and speech 13
7. Deep learning 14
8. Other features: chatbot/softbot 14

Conclusion 15

References **16**

Annexure A: Grammarly report ..... **A – 1**

Table of figures

Figure 1: Data files used by PREDICT CARE 10
Figure 2: Saved trained models 11
Figure 3: Time-series evaluation of the six scenarios 12
Figure 4: Repository structure and main scripts 13
Figure 5: The local language assistant running 14
Figure 6: Deep-learning training function 14

DOCUMENTATION ASPECTS

1. PREDICT CARE's relevance to the theme

PREDICT CARE responds to the theme of an artificial intelligence (AI) solution for industries by applying AI to the healthcare industry, where hospitals and clinics must decide quickly how urgent each patient is while using limited doctors, time and space. Many healthcare facilities in developing countries still rely on observation and manual records to manage patient flow, so the benefits of the fourth industrial revolution (4IR) are not yet reaching the people on the ground. PREDICT CARE is a Python-based decision-support prototype that uses machine learning, deep learning, natural language processing (NLP), time-series forecasting and a local conversational assistant. It predicts the triage level of a patient, based on the Korean Triage and Acuity Scale (KTAS) (Moon et al., 2019), and the expected waiting times from patient and hospital data. It is relevant to the theme because it turns data that healthcare facilities already hold into practical information that helps staff prioritise patients, plan patient flow and use resources more effectively. PREDICT CARE supports healthcare professionals and managers; it does not replace them or make medical diagnoses.

2. Business objectives

a. Main business objective

To improve patient management and operational efficiency in healthcare facilities by using AI-driven predictions to support patient prioritisation, waiting-time management, patient-flow planning and resource allocation.

b. Specific business objectives

1. Improve patient prioritisation

To analyse available patient information and provide insights that can assist healthcare staff in identifying patients who may require more urgent attention.

2. Improve waiting-time management

To analyse patterns in patient demand and waiting times in order to help healthcare facilities identify periods where delays and increased patient demand may occur.

3. Improve patient-flow management

To provide data-driven insights that can assist healthcare facilities in managing the movement of patients through healthcare services and reducing unnecessary delays.

4. Support resource allocation

To provide information about expected patient demand that can assist healthcare managers in planning the use of available staff and healthcare resources.

5. Support data-driven decision-making

To transform available healthcare data into meaningful predictions and insights that can assist healthcare staff and managers in making operational decisions.

6. Improve operational efficiency

To use AI to identify patterns in healthcare operations that may assist facilities in planning their services and making more effective use of available resources.

3. Business success criteria

The success of PREDICT CARE will be assessed according to whether the proposed solution is able to provide useful and reliable information that addresses the identified healthcare-management challenges.

- The system should be able to successfully process and analyse the selected healthcare dataset.
- The machine learning model should produce predictions with an acceptable level of accuracy based on the evaluation measures selected for the solution.

- The system should identify meaningful patterns in patient demand, waiting times or other selected variables within the dataset.
- The predictions generated by the system should be understandable and relevant to the intended healthcare-management problem.
- The system should provide information that could assist with patient prioritisation and patient-flow planning.
- The system should demonstrate how AI-generated insights could support better resource planning and operational decision-making.
- The proposed solution should successfully demonstrate the application of Python and appropriate AI/machine-learning techniques to a healthcare problem.
- The final prototype should provide a clear demonstration of how the proposed system could benefit a healthcare organisation.

These criteria will allow the group to evaluate not only whether the AI model functions technically, but also whether the proposed solution addresses the business problem it was designed to solve.

4. Business background

Healthcare facilities are required to manage large numbers of patients while making effective use of limited resources such as healthcare professionals, consultation areas and available time. Patient demand can vary throughout the day and over different periods, which can contribute to congestion, long waiting times and difficulties in managing patient flow.

Traditional approaches to patient and operational management may rely heavily on manual processes and historical information. This can make it difficult for healthcare facilities to identify patterns in patient demand and anticipate periods when additional pressure on services may occur.

PREDICT CARE is proposed as an AI-based solution to assist healthcare facilities in using their available data more effectively. By analysing historical and relevant patient-management data, the system can identify patterns and

generate predictions that may assist healthcare staff and managers with patient prioritisation, patient-flow planning and resource allocation.

The solution therefore focuses not only on the use of AI technology but also on how AI can provide practical benefits to an organisation within the healthcare industry.

5. Requirements

a. Data requirements

PREDICT CARE depends on two healthcare datasets: an emergency-service triage dataset and a hospital simulation dataset. The data must meet the following requirements so that the models can learn from it reliably.

- The triage dataset must contain the input fields required by the selected model and the KTAS_expert target, because the classifier learns by comparing the inputs with the expert triage category.
- The patient-flow dataset must contain the operational input variables and the target time outcomes required by each regression task, so that the models can learn how priority, staffing and hospital scenario affect waiting and total time.
- Data must be prepared in a consistent format and checked for missing, invalid or inconsistent values, because errors in the data lead to unreliable predictions.
- Training and evaluation data must be separated so that evaluation is performed on data not used to fit the model; otherwise the results would look better than the real performance.
- The data used in demonstrations must be appropriate for an educational prototype, and personal or sensitive information must be handled responsibly.

b. Functional requirements

- The solution must accept or process the input fields required by the triage prediction component, so that a user can supply patient information.
- The solution must generate a predicted KTAS_expert category from the triage model when valid input data is provided.
- The solution must process hospital operational inputs and generate predictions for the supported waiting-time or total-time targets.
- The solution should make prediction outputs available through the intended interface or prediction service, subject to the components that are complete and tested.
- The solution should communicate that its outputs are estimates for decision support and not clinical diagnoses or instructions, so that users do not misuse them.

c. Non-functional requirements

- The solution should produce consistent outputs when supplied with the same valid inputs and model version, so that users can trust the results.
- Input data should be checked for required fields and acceptable values before a prediction is made, to prevent errors and unreliable output.
- The code should be organised so that the model components can be maintained and tested separately.
- The solution should provide understandable output to its intended users, who are healthcare staff and managers and not programmers.
- Data should be handled securely and used only for the purposes permitted for the solution.

Together, these requirements make sure that PREDICT CARE produces reliable, understandable and responsibly used decision-support information.

6. Constraints

- **Prototype scope:** PREDICT CARE is an academic prototype and is not clinically validated or certified as a medical system. It demonstrates technical feasibility and has not been shown to improve patient outcomes or hospital efficiency.
- **Simulated and limited data:** the patient-flow dataset is simulation-based, and the time-series analysis uses only 111 records across six scenarios, so results should not automatically be applied to every real hospital.
- **Data privacy:** healthcare information can be sensitive, so personally identifiable patient information must not be exposed, and publicly available or anonymised data should be used. In South Africa, the processing of personal information is regulated by the Protection of Personal Information Act 4 of 2013 (POPIA) (Republic of South Africa, 2013).
- **Time, skills and computing resources:** the solution had to be completed within the time available, by students, using the computers and software available to the group, which limits how complex the models can be.
- **Uneven model performance:** the NLP triage model performs poorly for KTAS levels 2 to 4, and the medication-wait model performs worse than an always-zero baseline, so these components need further improvement.
- **Internet dependency:** speech input uses Google Speech Recognition and therefore requires an internet connection; typed input remains available.

7. Risks

Risk	Possible Impact	Mitigation
Poor-quality or incomplete data	The model may produce unreliable predictions.	Clean the dataset and identify/remove or appropriately handle missing and inconsistent data.
Insufficient dataset size	The model may not learn patterns effectively.	Select a suitable dataset and evaluate whether it contains sufficient relevant observations.
Model overfitting	The model may perform well on training data but poorly on unseen data.	Separate training and testing data and use appropriate model-evaluation techniques.
Inaccurate predictions	Incorrect predictions could reduce the usefulness of the system.	Evaluate the model using appropriate performance measures and compare suitable algorithms where possible.
Data privacy concerns	Sensitive patient information could be exposed.	Use anonymised or publicly available data and avoid including personally identifiable information.
Technical difficulties	Problems with Python, libraries or the development environment could delay development.	Test the development environment early and maintain backups of all files.
Limited development time	The group may not complete all planned features.	Prioritise the core AI functionality and avoid unnecessary features.
Misinterpretation of AI results	Users may treat predictions as medical decisions.	Clearly communicate that PREDICT CARE is a decision-support tool and does not replace professional medical judgement.

Table 1: Risk table

8. Tools and techniques

PREDICT CARE will use Python as the primary programming language.

a. Python

Python will be used to develop the data-processing, machine-learning and prediction components of PREDICT CARE.

b. Data cleaning and preprocessing

Data preprocessing techniques will be used to prepare the dataset before it is supplied to the machine-learning model. This may include:

- Handling missing values.
- Removing duplicate records.
- Correcting inconsistent data.
- Converting data into suitable formats.
- Selecting relevant variables.
- Preparing categorical and numerical data for modelling.

c. Exploratory data analysis

Exploratory Data Analysis (EDA) will be used to understand the dataset and identify patterns, relationships, trends and unusual values before developing the machine-learning model.

d. Machine learning

A suitable machine-learning algorithm will be selected based on the type of prediction problem being addressed. The model will be trained using the prepared dataset and tested using unseen data.

The selection of the final algorithm will be based on its suitability for the selected problem and its performance during evaluation.

e. Model evaluation

The machine-learning model will be evaluated using appropriate performance measures. The evaluation method will depend on whether the final prediction problem is treated as a classification or regression problem.

The purpose of model evaluation will be to determine whether the model can produce reliable predictions and whether improvements or alternative algorithms are required.

f. Data visualisation

Graphs and other visualisations may be used to present patterns and results from the dataset. Visualisation will help users understand trends in patient demand, waiting times and other relevant variables.

g. GitHub

GitHub will be used to store and manage the group's work.

9. Problem definition

Healthcare facilities must assess how urgently each patient needs care while also managing patient flow and limited resources such as doctors and consultation time. Patient arrivals, patient priority, staffing levels and service times all influence how long patients wait. Because patient numbers vary, managers often rely on observation and manual records, which makes it difficult to anticipate congestion, understand where delays occur or judge how many staff are needed. Long waiting times place pressure on staff and may delay patients who need attention sooner. Facilities hold triage and operational data, but without analysis this data does not give staff or managers usable, forward-looking information about patient urgency, waiting times and workload.

THEORETICAL ASPECTS

1. Machine learning approach

PREDICT CARE has adopted a supervised machine learning approach to support patient triage and hospital-flow prediction. The main algorithms implemented include Random Forest Classification for predicting KTAS triage levels from structured patient information, Random Forest Regression for estimating doctor waiting times and other patient-flow measures, Neural Networks for learning patterns in patient data to classify KTAS levels, Term Frequency–Inverse Document Frequency (TF-IDF) and text classification for converting written chief complaints into numerical features for NLP-based triage prediction, and Time-Series Forecasting, where Simple Exponential Smoothing (SES) and Holt's Trend method are evaluated to forecast trends across hospital simulation scenarios. The Random Forest algorithm follows Breiman (2001), the models are built with the scikit-learn library (Pedregosa et al., 2011), and the forecasting methods follow Hyndman and Athanasopoulos (2021).

These approaches were selected to address classification, regression, text analysis, and forecasting tasks within the system.

2. Data

PREDICT CARE employed the use of two datasets. The first, ktas_cleaned.csv, contains patient demographics, arrival information, vital signs, symptoms, and expert-assigned KTAS levels, supporting structured triage classification and deep learning. The second, patient_flow_cleaned.csv, contains 60,000 simulation-based records used to train waiting-time and patient-flow models.

The structure of ktas_cleaned.csv corresponds to the publicly available Emergency Service – Triage Application dataset (ilkeryildiz, n.d.), which is based on the study by Moon et al. (2019). The data was cleaned and preprocessed using techniques such as missing-value imputation, categorical

encoding, numerical scaling where required, text cleaning, and TF-IDF vectorisation. It should be noted that the simulation-based data may not fully represent real-world hospital conditions.

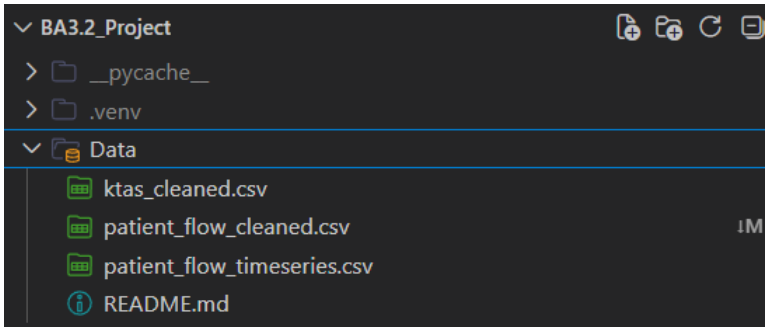


Figure 1: Data files used by PREDICT CARE

3. Model development and evaluation

Models were evaluated on held-out test data using metrics suited to each task. Classification was assessed using accuracy, precision, recall, F1-score, and confusion matrices, while regression was assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

Model	**Algorithm**	**Test results**
Structured KTAS triage	Random Forest Classifier	82.73% accuracy
Deep-learning triage	Neural network	84.60% accuracy
NLP triage	TF-IDF-based text classifier	60.19% accuracy; macro F1 0.4286
Doctor waiting time	Random Forest Regression	MAE 26.26 minutes; R^2 0.9812
NLP pipeline waiting time	Regression models	Doctor-wait MAE 26.53 minutes; total-time MAE 27.35 minutes

Table 2: Model test results

The NLP triage model performs poorly for KTAS levels 2–4, and the medication-wait model performs worse than an always-zero baseline on overall MAE. These weaknesses require further improvement.

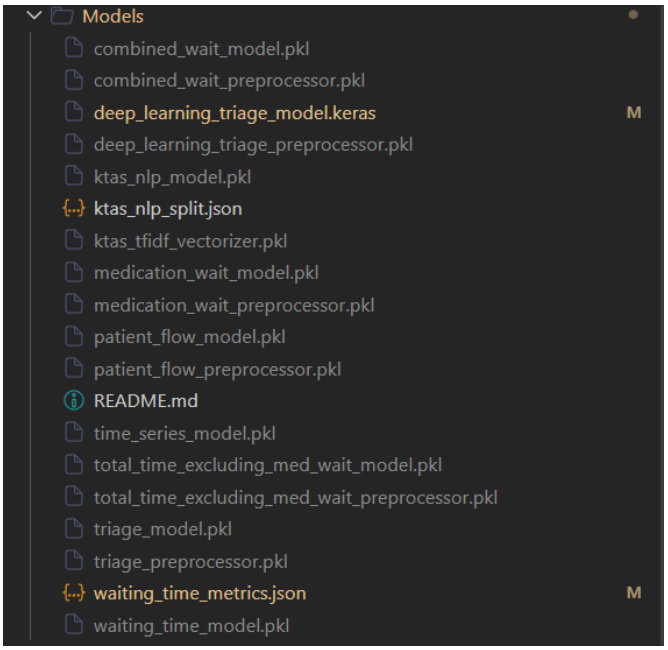


Figure 2: Saved trained models

4. Time-series analysis on data

The time_series.py script analyses 111 records across six hospital simulation scenarios involving different staffing levels and patient-arrival patterns. It compares SES with Holt's Trend method, evaluating forecast errors using MAE and RMSE. SES was selected for all six scenarios because it performed better in these tests. The small, scenario-based dataset limits generalisation to real hospitals.



Figure 3: Time-series evaluation of the six scenarios

5. Solution techniques and main scripts

Script	Technique and purpose
healthcare_ai.py	Uses Random Forest classification for structured KTAS prediction and Random Forest regression for doctor waiting time, combined waiting time, and total-time estimates. It also trains a medication-wait model.
deep_learning.py	Uses a neural network with encoded patient features, class weighting, and early stopping to predict KTAS levels.
NLP/train_ktas_model.py	Cleans chief-complaint text, combines word-level and character-level TF-IDF features, and trains a text classifier to predict KTAS levels.
NLP/train_waiting_time_model.py	Trains regression models to estimate doctor waiting time and total hospital time from patient-flow features.
time_series.py	Compares SES and Holt's Trend forecasting methods across six simulation scenarios.
NLP/pipeline.py	Loads saved NLP and waiting-time models, processes requests, routes predictions, validates inputs, and returns estimated times.
local_language_engine.py	Uses TF-IDF, Logistic Regression, intent classification, predefined responses, and regular-expression rules to interpret user requests and identify selected emergency warning patterns.

Script	Technique and purpose
main_menu.py	Provides a command-line interface for KTAS and waiting-time predictions, with typed input and speech recognition.
Main.py	Checks model and file availability and runs integration checks for the system components.

Table 3: Main scripts and techniques

Potential improvements after completion include adding representative real-world data, tuning hyperparameters, improving NLP performance for underperforming KTAS classes, comparing against suitable baselines, and performing independent validation.

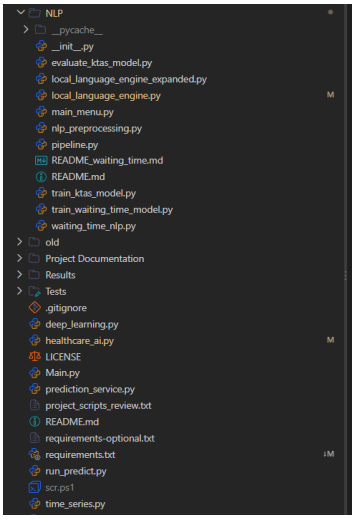


Figure 4: Repository structure and main scripts

6. Natural language processing and speech

The NLP component processes written chief complaints using text cleaning and combined word-level and character-level TF-IDF features to predict KTAS triage levels. The local language assistant uses TF-IDF and Logistic Regression for intent classification, predefined responses, and rule-based emergency warnings. Speech synthesis is implemented using pyttsx3, allowing users to choose between spoken responses and text-only output at startup. Speech input is supported through Google Speech Recognition in main_menu.py, requiring an internet connection.

```
(.venv) PS C:\Users\maxfi\Desktop\BA3.2_Project\BA3.2_Project> python nlp/local_language_engine.py
============================================================
Predict-Care AI — Local Language Engine
No Ollama or cloud model required.
Intent classifier: available
Unified pipeline: available
Speech synthesis: available (voice on by default)
Type /help for commands, /voice on, /voice off, or /quit to exit.
============================================================

You: Hello

Assistant: Hello! I am the Predict-Care AI local assistant. You can ask me about KTAS, describe a patient for an experimen
tal triage prediction, or ask a waiting-time question. For example: 'a patient with chest pain', or 'high risk, 2 doctors,
 peak, how long will I wait?'.

You:
```

Figure 5: The local language assistant running

7. Deep learning

PREDICT CARE applies Deep Learning techniques through the deep_learning.py script, which uses a neural network implemented with Keras (Chollet, 2015) and the PyTorch backend to predict expert KTAS levels from structured patient features. It applies numerical preprocessing, one-hot encoding, class weighting, and early stopping. The model achieved 84.60% test accuracy and saves the trained network and preprocessor for later use. This is an academic prototype, not a clinically validated model.

deep_learning.py > train_deep_learning_model

```python
# ============================================================
# TRAIN MODEL
# ============================================================

Windsurf: Refactor | Explain | ×
def train_deep_learning_model(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
):
    """Here I am training the neural network with class balancing."""
    model = build_deep_learning_model(
        input_size=X_train.shape[1],      You, 4 weeks ago • Model for 2 data
        number_of_classes=len(KTAS_CLASSES),
    )

    # Here I am giving additional weight to less common KTAS classes.
    class_indices = np.unique(y_train)
    weights = compute_class_weight(
        class_weight="balanced",
        classes=class_indices,
        y=y_train,
    )
    class_weights = {
        int(class_index): float(weight)
        for class_index, weight in zip(class_indices, weights)
    }

    callbacks = [
        keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=8,
            restore_best_weights=True,
            verbose=1,
        )
    ]
```

Figure 6: Deep-learning training function

8. Other features: chatbot/softbot

The local language assistant recognises intents such as greetings, help requests, KTAS explanations, triage prediction, and waiting-time questions. It

routes supported requests to the prediction pipeline, maintains limited conversational context, and uses regular-expression rules to flag selected emergency warning patterns. It does not use a generative large language model.

Conclusion

PREDICT CARE combines traditional machine learning, deep learning, NLP, time-series forecasting, speech input, and a local conversational assistant to support healthcare triage and patient-flow planning. Initial results demonstrate technical feasibility, but further validation and model improvement are necessary. The system is an academic decision-support prototype and must not replace professional clinical assessment or independently determine patient care.

References

1. Books and learning resources

- Brownlee, J. (2021) Machine Learning Mastery with Python: Understand Your Data, Create Accurate Models and Work Projects End-to-End. Independently published. Available at: https://books.google.com/books?id=iOI-zwEACAAJ
- Géron, A. (2017) Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. 1st edn. Sebastopol, CA: O'Reilly Media.
- Sarkar, D., Bali, R. and Sharma, T. (2018) Practical Machine Learning with Python: A Problem-Solver's Guide to Building Real-World Intelligent Systems. Berkeley, CA: Apress. Available at: https://github.com/dipanjanS/practical-machine-learning-with-python
- Rishabh-creator601 (n.d.) Books: Natural Language Processing (NLP). GitHub repository. Available at: https://github.com/Rishabh-creator601/Books/tree/master/NLP
- Brownlee, J. (2016) Machine Learning Mastery with Python: Understand Your Data, Create Accurate Models and Work Projects End-To-End. Version 1.4.
- Lee, W-M. (2019) Python Machine Learning. Indianapolis: John Wiley & Sons.

2. Machine learning and artificial intelligence

- GeeksforGeeks (2026) Introduction to Machine Learning. Available at: https://www.geeksforgeeks.org/machine-learning/introduction-machine-learning/
- Tutorials Point (2016) AI with Python. Available at: https://www.tutorialspoint.com/artificial_intelligence_with_python/
- Scikit-learn (n.d.) RandomForestClassifier. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- Scikit-learn (n.d.) train_test_split. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- Scikit-learn (n.d.) TfidfTransformer. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html
- Keras (n.d.) The Sequential Model. Available at: https://keras.io/guides/sequential_model/

3. Datasets used in Predict-Care AI

- İlker Yıldız (n.d.) Emergency Service Triage Application. Kaggle dataset. Available at: https://www.kaggle.com/datasets/ilkeryildiz/emergency-service-triage-application

- Juneminazuki (n.d.) Hospital Queue and Patient Flow Simulation. Kaggle dataset. Available at: https://www.kaggle.com/datasets/juneminazuki/hospital-queue-and-patient-flow-simulation

4. Speech recognition and voice processing

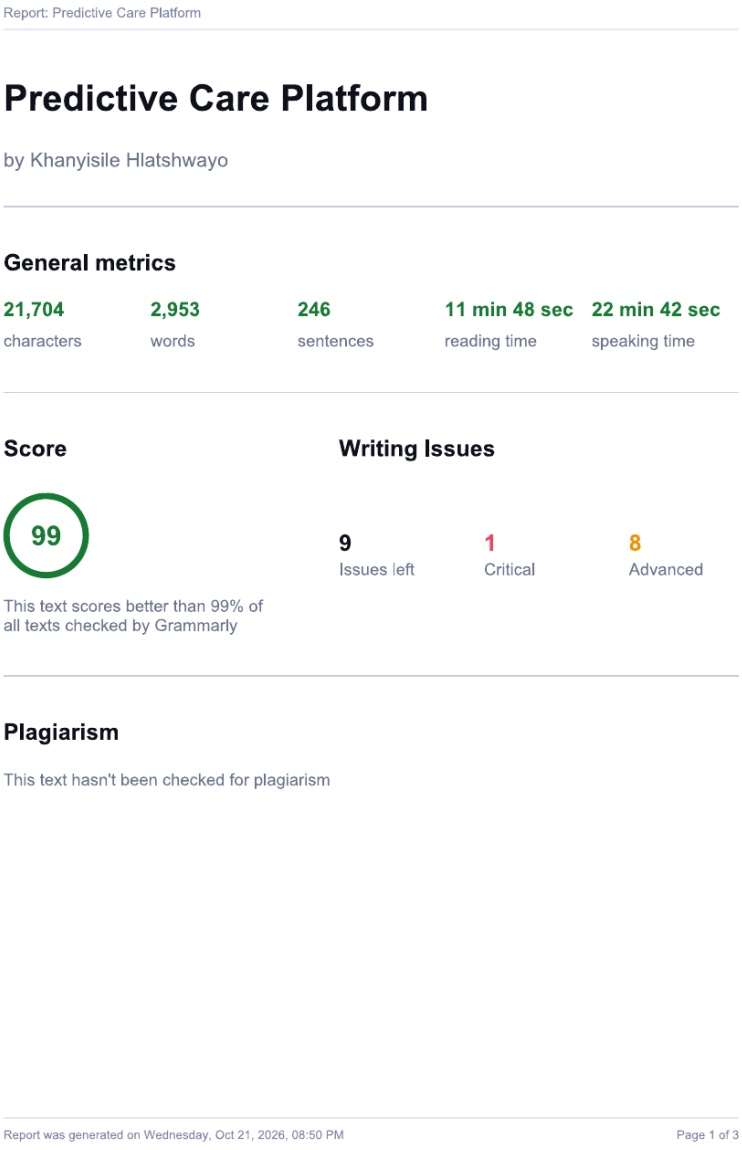
- SpeechRecognition (n.d.) SpeechRecognition: Library for Performing Speech Recognition. Python Package Index (PyPI). Available at: https://pypi.org/project/SpeechRecognition/

5. Additional software and development resources

- Python Software Foundation (n.d.) Python Documentation. Available at: https://docs.python.org/3/
- Pandas (n.d.) Pandas Documentation. Available at: https://pandas.pydata.org/docs/
- TensorFlow (n.d.) TensorFlow Documentation. Available at: https://www.tensorflow.org/learn

Group Github Repo: https://github.com/ME-Moeketsi/BA3.2_Project

Annexure A: Grammarly report



Report: Predictive Care Platform

Predictive Care Platform

by Khanyisile Hlatshwayo

General metrics

21,704	2,953	246	11 min 48 sec	22 min 42 sec
characters	words	sentences	reading time	speaking time

Score

99

This text scores better than 99% of all texts checked by Grammarly

Writing Issues

9	1	8
Issues left	Critical	Advanced

Plagiarism

This text hasn't been checked for plagiarism

Report was generated on Wednesday, Oct 21, 2026, 08:50 PM

Page 1 of 3

Writing Issues

2	**Delivery**	
2	Inappropriate colloquialisms	
5	**Clarity**	
5	Paragraph can be improved	
2	**Correctness**	
1	Improper formatting	
1	Closing punctuation	

Unique Words

Measures vocabulary diversity by calculating the percentage of words used only once in your document

14%

unique words

Rare Words

Measures depth of vocabulary by identifying words that are not among the 5,000 most common English words.

38%

rare words

Word Length

Measures average word length

6.1

characters per word

Annexure A – 2

Sentence Length

Measures average sentence length

12

words per sentence

Annexure A – 3